

Sender Algorithm Customization

Document #: P2999R0
Date: 2023-10-14
Project: Programming Language C++
Audience: LEWG Library Evolution
Reply-to: Eric Niebler
<eric.niebler@gmail.com>

Contents

- [1 Introduction](#)
- [2 The Issue](#)
- [3 Proposed Design](#)
 - [3.1 Features and rationale](#)
 - [3.1.1 Dispatching via domain tags](#)
 - [3.1.2 Late \(sender/receiver connection-time\) customization](#)
 - [3.1.3 Early \(sender construction-time\) customization](#)
 - [3.1.4 Decomposable senders](#)
 - [3.2 Summary of proposed changes](#)
- [4 Implementation Experience](#)
- [5 Proposed Wording](#)
- [6 References](#)

§ 1 Introduction

This paper proposes some design changes to P2300 to address some shortcomings in how algorithm customizations are found.

§ 2 The Issue

The essence of the issue is this:

Many senders do not know on what execution context they will complete, so using solely that information to find customizations (as P2300R7 does) is unsatisfactory.

In [P2300R7], the sender algorithms (then, `let_value`, etc) are customization point objects that internally dispatch via `tag_invoke` to the correct algorithm implementation. Each algorithm has a default implementation that is used if no custom implementation is found.

Custom implementations of sender algorithms are found by asking the predecessor sender for its completion scheduler and using the scheduler as a tag for the purpose of tag dispatching. A *completion scheduler* is a scheduler that refers to the execution context on which that sender will complete.

A typical sender algorithm like then might be implemented as follows:

```
/// @brief A helper concept for testing whether an algorithm customization
/// exists
template <class AlgoTag, class SetTag, class Sender, class... Args>
concept has-customization =
    requires (Sender sndr, Args... args) {
        tag_invoke(AlgoTag(),
            get_completion_scheduler<SetTag>(get_env(sndr)),
            std::forward<Sender>(sndr),
            std::forward<Args>(args)...);
    };

/// @brief The tag type and the customization point object type for the
/// `then` sender algorithm
struct then_t {
    template <sender Sender, class Fun>
        requires /* requirements here */
    auto operator()(Sender&& sndr, Fun fun) const
    {
        // If the predecessor sender has a completion scheduler, and if we can use
        // the completion scheduler to find a custom implementation for the `then`
        // algorithm, dispatch to that. Otherwise, dispatch to the default `then`
        // implementation.
        if constexpr (has-customization<then_t, set_value_t, Sender, Fun>)
        {
            auto&& env = get_env(sndr);
            return tag_invoke(*this,
                get_completion_scheduler<set_value_t>(env),
                std::forward<Sender>(sndr),
                std::move(fun));
        }
        else
        {
            return then-sender<Sender, Fun>(std::forward<Sender>(sndr), std::move(fun));
        }
    }
};

inline constexpr then_t then {};
```

This scheme has a number of shortcomings:

1. A simple sender like `just(42)` does not know its completion scheduler. It completes on the execution context on which it is started. That is not known at the time the sender is constructed, which is when we are looking for customizations.
2. For a sender like `on( sched, then(just(), fun) )`, the nested then sender is constructed before we have specified the scheduler, but we need the scheduler to dispatch to the correct customization of then. How?

3. A composite sender like `when_all(sndr1, sndr2)` cannot know its completion scheduler in the general case. Even if `sndr1` and `sndr2` both know their completion schedulers – say, `sched1` and `sched2` respectively – the `when_all` sender can complete on *either* `sched1` or `sched2` depending on which of `sndr1` and `sndr2` completes last. That is a dynamic property of the program’s execution, not suitable for finding an algorithm customization.

In cases (1) and (2), the issue is that the information necessary to find the correct algorithm implementation is not available at the time we look for customizations. In case (3), the issue is that the algorithm semantics make it impossible to know statically to what algorithm customization scheme to dispatch.

The issue described in (2) above is particularly pernicious. Consider these two programs (where `ex::` is a namespace alias for `std::execution`):

Good	Bad
<pre> my::thread_pool_scheduler sch = /*...*/; // Describe some bulk work on a thread pool auto work = ex::transfer_just(sch, data) ex::bulk(data.size(), [](int i, auto& data) { ++data[i]; }); // Execute the work std::this_thread::sync_wait(std::move(work)); </pre>	<pre> my::thread_pool_scheduler sch = /*...*/; // Describe some bulk work auto work = ex::just(data) ex::bulk(data.size(), [](int i, auto& data) { ++data[i]; }); // Execute the bulk work on a thread pool std::this_thread::sync_wait(ex::on(sch, std::move(work))); </pre>

These two programs *should* be equivalent, but they are not. The author of the `thread_pool_scheduler` gave it a custom bulk implementation by defining:

```

namespace my {
    // customization of the bulk algorithm for the thread_pool_scheduler:
    template <ex::sender Sender, std::integral Shape, class Function>
    auto tag_invoke(ex::bulk_t,
        thread_pool_scheduler sched,
        Sender&& sndr,
        Shape shape,
        Function fun) {
        /*...*/
    }
}

```

This overload is found only when the bulk sender’s predecessor completes on a `thread_pool_scheduler`.

In the code to the right, however, the predecessor of the bulk operation is `just(data)`, a sender that does not know where it will complete. As a result, the above customization of the bulk algorithm will not be found, and the bulk operation will execute serially on a single thread in the thread pool. That’s almost certainly *not* what the programmer intended.

This is clearly broken and badly in need of fixing.

Note: On the need for async algorithms customization

It is worth asking why async algorithms need customization at all. After all, the classic STL algorithms need no customization; they dispatch using a fixed concept hierarchy to a closed set of possible implementations.

The reason is because of the open and continually evolving nature of execution contexts. There is little hope of capturing every salient attribute of every interesting execution model – CPUs, GPUs, FPGAs, etc., past, present, and future – in a fixed ontology around which we can build named concepts and immutable basis operations. Instead we do the best we can and then hedge against the future by making the algorithms customizable.

§ 3 Proposed Design

§ 3.1 Features and rationale

This section describes at a high level the salient features of the proposed design for sender algorithm customization, and their rationale.

§ 3.1.1 Dispatching via domain tags

As described above, the `when_all` sender doesn't know its completion scheduler, so we cannot use the completion scheduler to find the `when_all` customization. Instead, we can use an abstract tag type – a so-called *domain* – to dispatch to the correct customizations. As long as `when_all`'s child senders all share a domain, we can know what set of algorithm customizations to use.

This paper proposes the addition of a forwarding `get_domain` query, and that the domain is used together with the algorithm tag to dispatch to the correct algorithm implementation.

Additionally, we proposed that the `when_all` algorithm only accepts a set of senders when they all share a common domain. Likewise for `let_value` and `let_error`, we require that there is only one possible domain on which their senders may complete.

§ 3.1.2 Late (sender/receiver connection-time) customization

As described above, the sender algorithm customization points don't have all the information they need to dispatch to the correct algorithm implementation in all cases. The solution is to look again for a customization when all the information is available. That happens when the sender is connect-ed to a receiver.

This paper proposes the addition of a `transform_sender` customization point that is called by the connect customization point to transform a sender prior to connecting it with the receiver. *In this sense, it is precisely analogous to the `await_transform` customization used by coroutines to transform an expression prior to `co_await`-ing it.*

§ 3.1.3 Early (sender construction-time) customization

We can use `transform_sender` for early customization as well as late. The benefit of doing this is that only one set of customizations needs be written for each domain, rather than two (early and late).

This paper proposes that each algorithm constructs a default sender that implements the default behavior for that algorithm. It then passes that sender to `transform_sender` along with the sender's domain. The result of `transform_sender` is what the algorithm returns.

Some algorithms are required to do work eagerly in their default implementation (*e.g.*, `split`, `ensure_started`). These algorithms must first create a dummy sender to pass to `transform_sender`. The “default” domain, which is used when no other domain has been specified, can transform these dummy senders and do their eager work in the process. The same mechanism is also useful to implement customizable sender algorithms whose default implementation merely lowers to a more primitive expression (*e.g.* `transfer(s, sch)` becomes `schedule_from(sch, s)`, and `transfer_just(sch, ts...)` becomes `just(ts...) | transfer(sch)`).

To permit third parties to author customizable sender algorithms that do eager work in their default implementations, the mechanism by which the default domain finds the default sender transformations shall be specified.

§ 3.1.4 Decomposable senders

For the `transform_sender` customization point to be useful, we need a way to access the constituent pieces of a sender and re-assemble it from (possibly transformed) pieces. Senders, like coroutines, generally begin in a “suspended” state; they merely curry their algorithm's arguments into a subsequent call to connect. These “suspended” senders are colloquially known as *lazy* senders.

Each lazy sender has an associated algorithm tag, a (possibly empty) set of auxiliary data and a (possibly empty) set of child senders; *e.g.*, the sender returned from `then(snd, fun)` has `then_t` as its tag, the set `[fun]` as its auxiliary data, and `[snd]` as its set of child senders, while `just(42, 3.14)` has `just_t` as its tag, `[42, 3.14]` as its data set and `[]` as its child set.

This paper proposes to use structured bindings as the API for decomposing a lazy sender into its tag, data, and child senders:

```
auto&& [tag, data, ...children] = sndr;
```

[P1061R5], currently in Core wording review for C++26, permits the declaration of variadic structured bindings like above, making this syntax very appealing.

Not all senders are required to be decomposable, although all the “standard” lazy senders shall be. There needs to be a syntactic way to distinguish between decomposable and non-decomposable senders (decomposable senders subsuming the sender concept).

There is currently no trait for determining whether a type can be the initializer of a structured binding. However, EWG has already approved [P2141R1] for C++26, and with it such a trait could be built, giving us a simple way to distinguish between decomposable and non-decomposable senders.

If P2141 is not adopted for C++26, we will need some other syntactic way to opt-in. One possibility is to require that the sender type's nested `is_sender` type shall have some known, standard tag type

as a base class to signify that that sender type can be decomposed.

Note: After decomposing a sender, it is often desirable to re-compose it from its modified constituents. No separate API for reconstituting senders is necessary though. It is enough to construct a decomposable sender of some arbitrary type and then pass it to `transform_sender` with the appropriate domain tag.

§ 3.2 Summary of proposed changes

In condensed form, here are the changes this paper is proposing:

1. Add a `default_domain` type for use when no other domain is determinable.
2. Add a new `get_domain(env) -> domain-tag` forwarding query.
3. Add a new, non-customizable `transform_sender(domain, sender [, env]) -> sender` API. It will be used for both early customization (at sender construction-time) and late customization (at sender/receiver connection-time).

Early customization:

- called from within each sender algorithm’s customization point object
- replaces the current mechanism of tag-dispatching to a sender factory function using the completion scheduler as a tag
- called without an environment argument
- `domain` is derived from the sender by trying the following in order:
 1. `get_domain(get_env(sender))`
 2. `get_domain(get_completion_scheduler<completion-tag>(get_env(sender)))`, where `completion-tag` is one of `set_value_t`, `set_error_t`, or `set_stopped_t` depending on the algorithm
 3. `default_domain()`

Late customization:

- called from the connect customization point object before tag-dispatching with `connect_t` to `tag_invoke`
- called with the receiver’s environment
- `domain` is derived from the receiver by trying the following in order:
 1. `get_domain(get_env(receiver))`
 2. `get_domain(get_scheduler(get_env(receiver)))`
 3. `default_domain()`

`transform_sender(domain, sender [, env])` returns the first of these that is well-formed:

- `domain.transform_sender(sender [, env])`
- `default_domain().transform_sender(sender [, env])`

— *sender*

4. The standard, “lazy” sender types (i.e., those returned from sender factory and adaptor functions) return sender types that are decomposable using structured bindings into its [*tag*, *data*, ...*children*] components.
5. A call to the `when_all` algorithm should be ill-formed unless all of the sender arguments have the same domain type (as determined for senders above). The resulting `when_all` sender should publish that domain via the sender’s environment.
6. The `on(sch, sndr)` algorithm should be specified in terms of transfer so as to pick up any late customization of the transfer algorithm. (This amounts to changing `schedule(sch)` to `transfer_just(sch)` in [exec.on]/3.2.2.). Additionally, it should replace the domain in the receiver’s environment with the domain of `sch`.
7. The sender factories `just`, `just_error`, and `just_stopped` need their tag types to be specified. Name them `just_t`, `just_error_t`, and `just_stopped_t`.
8. In the algorithm `let_value(sndr, fun)`, if the predecessor sender `sndr` has a completion scheduler for `set_value`, then the receiver connected to the secondary sender (the one returned from `fun` when called with `sndr`’s results) shall expose that scheduler as the current scheduler of the receiver’s environment.

In other words, if the predecessor sender `sndr` completes with values `vs...`, then the result of `fun(vs...)` will be connected to a receiver `r` such that `get_scheduler(get_env(r))` is equal to `get_completion_scheduler<set_value_t>(get_env(sndr))`.

The same is true also of the domain query: `get_domain(get_env(r))` is equal to the domain of the predecessor sender as computed by the steps in (2) above.

So for `let_value`, likewise also for `let_error`, using `set_error_t` when querying for the predecessor sender’s completion scheduler. (`let_stopped` needs no modification because the nullary function passed to `let_stopped` can only have a single return type; hence there is only one secondary sender type and one domain to consider.)

9. The `schedule_from(sched, sndr)` algorithm should return a sender `s` such that `get_domain(get_env(s))` is equal to `get_domain(sched)`.
10. The following customizable algorithms, whose default implementations must do work before returning the result sender, will have their work performed in overloads of `default_domain::transform_sender`:

— `split`

— `ensure_started`

11. The following customizable algorithms, whose default implementations are trivially expressed in terms of other more primitive operations, will be lowered into their primitive forms by overloads of `default_domain::transform_sender`:

— `transfer`

— `transfer_just`

- `transfer_when_all`
- `transfer_when_all_with_variant`
- `when_all_with_variant`

12. In the algorithm `let_value(snd, fun)`, all of the sender types that the input function `fun` might return must all have the same domain; otherwise, the call to `let_value` is ill-formed. The resulting `let_value` sender will report that as its domain. Likewise for `let_error` and `let_stopped`.
13. Sender consuming algorithms `start_detached` and `sync_wait` will continue to dispatch to `tag_invoke` customizations using the algorithm tag and the input sender's domain as tags for the purpose of tag dispatching.

§ 4 Implementation Experience

Has it been implented? YES. The design changes herein proposed are implemented in the main branch of [\[stdexecgithub\]](#), the reference implementation. The bulk of the changes including `get_domain`, `transform_sender`, and the changes to connect have been shipping since [this commit](#) on August 3, 2023 which changed the `static_thread_pool` scheduler to use `transform_sender` to parallelize the bulk algorithm.

§ 5 Proposed Wording

TODO

§ 6 References

- [P1061R5] Barry Revzin, Jonathan Wakely. 2023-05-18. Structured Bindings can introduce a Pack.
<https://wg21.link/p1061r5>
- [P2141R1] Antony Polukhin. 2023-05-03. Aggregates are named tuples.
<https://wg21.link/p2141r1>
- [P2300R7] Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, and Bryce Adelstein Lelbach.
`std::execution`.
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2300r7.html>
- [stdexecgithub] stdexec.
<https://github.com/NVIDIA/stdexec>